

UAgent: Adversarial Co-evolution for Targeted Bug Revelation in Unit Testing

Yulong Fu
Xidian University
Xi'an, Shaanxi, China
ylfu@xidian.edu.cn

Yingtong Zhang
Xidian University
Xi'an, Shaanxi, China
zhangyingtong2023@163.com

Abstract

Unit testing remains crucial for software quality assurance, yet manually creating effective tests requires significant expertise. Only 17% of 82,447 GitHub projects contain test cases, revealing low adoption rates. While automated tools like Pynguin and EvoSuite reduce effort, their dependence on code coverage metrics is problematic due to weak correlation with defect detection.

Current limitations include: (1) Defects in LLM-generated tests, particularly for strongly-typed languages (2) Inadequate simulation of progressive attacks in traditional mutation testing

To address these, we propose UAgent - a novel adversarial co-evolution framework with: - TG Agent (Defender): Creates test suites via reflective closed-loop iteration - MG Agent (Attacker): Generates coupled mutations exposing defensive gaps - Resilience Evolution Mechanism: Enables continuous adaptation through agent competition

By transforming test generation into a dynamic adaptation process, UAgent establishes a new paradigm for resilient software validation, addressing fundamental gaps in current test automation methodologies through biologically-inspired learning mechanisms.

Key contributions include: - First application of adversarial ML principles to test generation - Framework-agnostic implementation for Python/Java ecosystems

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; • **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Formal security models**; *Software security engineering*.

Keywords

ML robustness, adversarial testing, mutation analysis, test automation, software resilience certification

ACM Reference Format:

Yulong Fu and Yingtong Zhang. 2025. UAgent: Adversarial Co-evolution for Targeted Bug Revelation in Unit Testing. In *Proceedings of the 2025 Workshop on Recent Advances in Resilient and Trustworthy Machine learning-driveN*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ARTMAN '25, Taipei, Taiwan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1909-7/25/10

<https://doi.org/10.1145/3733821.3763027>

systems (ARTMAN '25), October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3733821.3763027>

1 Introduction

Testing is an important yet expensive step in the software development lifecycle. Generating effective tests is a time-consuming and tedious task for developers. Nevertheless, the manual creation of high-quality unit tests demands significant domain expertise and laborious effort [5]. This substantial overhead often acts as a deterrent, leading developers to neglect test implementation entirely.

This trend is evidenced by empirical studies revealing that only 17% of 82,447 analyzed GitHub projects contained test files [26][11]. The automated generation of unit tests represents a pivotal research direction in Software Engineering (SE), primarily aimed at alleviating developers' testing burdens. High-quality automated tests play a crucial role in mitigating software failures by preemptively identifying defects. Current solutions for test generation encompass three principal paradigms: random test generators [25], dynamic symbolic execution, and search-based approaches [1, 9, 25]. Nevertheless, these methodologies exhibit significant limitations in assertion quality—often producing tests lacking assertions, containing over-generalized checks, or generating assertions that fail to adequately verify the Program Under Test's (PUT) specified behavior.

In light of these limitations, recent research has increasingly focused on harnessing machine learning-based code synthesis techniques for enhanced unit test generation. These efforts primarily leverage transformer-based Large Language Models (LLMs), such as Codex [4], which demonstrate strong capabilities in program synthesis. Key advancements include: - Bareiß et al. [2] applied few-shot learning with Codex for Java test generation, achieving performance comparable to feedback-directed methods on 18 Java methods. - ATHENATEST [27] fine-tuned the BART transformer model on Java functions and tests, reporting coverage equivalent to EvoSuite [10] across five Java projects. - Lemieux et al. [17] developed CODAMOSA, augmenting search-based testing with Codex-generated test prefixes (inputs without oracles). Evaluations on 27 Python projects showed it outperformed Pynguin [19] and Codex in coverage.

Despite promising preliminary results from these and other studies, they share two critical deficiencies: 1. **No explicit focus** on improving test suites' bug detection efficacy 2. **Persistent reliance on coverage metrics** despite their weak correlation with actual fault revelation. Given these limitations, recent research has increasingly focused on leveraging machine learning-based code synthesis techniques to enhance unit test generation. These efforts primarily utilize Transformer-based Large Language Models

(LLMs), such as Codex [4], which demonstrate strong capabilities in program synthesis. Notably, recent studies indicate that LLMs can generate specification-based oracles for weakly typed languages (e.g., JavaScript) achieving state-of-the-art performance [1, 9, 25]. However, for strongly typed languages like Java, oracle generation remains significantly challenging due to strict type constraints and complex execution semantics.

Contemporary testing methodologies face two interdependent constraints: Firstly, machine learning-based generators exhibit systematic deficiencies in test oracle construction, particularly for strongly-typed languages where correctness rates substantially lag behind weakly-typed alternatives. Secondly, conventional mutation testing techniques operate through static, single-pass defect injection, lacking capacity to model progressive adversarial sequences or evaluate system resilience dynamics under sustained pressure.

To overcome these limitations, we introduce UAgent—an adversarial co-evolution framework that reformulates test generation as a competitive learning process. The architecture comprises symbiotic agents: the TG Agent (Test Generator) employs a reflective closed-loop mechanism to iteratively refine test suites, while the MG Agent (Mutation Generator) concurrently develops sophisticated coupled mutations targeting emergent defensive vulnerabilities. Through structured competitive engagement, these agents drive continuous adaptation, where each iteration enhances the opponent’s capability space. The theoretical and engineering value of this framework is reflected in: For the first time, it applies the principles of Adversarial Machine Learning (Adversarial ML) to the field of test generation. Secondly, It supports the Python (HumanEval) and Java (Refactory) ecosystems without requiring fine-tuning or reliance on external tools like EvoSuite.

2 Background

Unit testing, which involves verifying individual code units in isolation, constitutes a foundational activity in software quality assurance [1]. It plays a critical role in modern development practices by enabling early defect detection and correction. Conventional automated test generation paradigms exhibit fundamental limitations that restrict their practical utility. Random and search-based methods [1, 9, 25] frequently produce tests with inadequate assertions—either entirely missing, excessively generalized, or semantically misaligned with specifications. Similarly, symbolic execution techniques derive assertions from the program’s current implementation behavior rather than intended functionality. Both approaches prioritize code coverage metrics despite extensive evidence demonstrating their weak correlation with actual fault detection capability, resulting in test suites that inadequately validate functional correctness.

The emergence of Large Language Models (LLMs) has introduced promising alternatives for test generation. While achieving state-of-the-art oracle generation for weakly-typed languages like JavaScript [7, 20, 24], these models face significant challenges with strongly-typed languages such as Java due to strict type constraints and complex execution semantics. Notable implementations like CODAMOSA [17] augment search-based testing but lack assertion oracles, while hybrid approaches like TOGLL [13] combine EvoSuite with fine-tuned LLMs yet inherit brittle test structures

[10] and suffer from limited generalization. Crucially, existing LLM-based methods exhibit a persistent deficiency: none explicitly focus on enhancing test suites’ bug detection efficacy, maintaining an over-reliance on coverage metrics that poorly predict fault revelation.

Mutation Testing (MT) provides a rigorous validation framework to address this limitation. Through injection of artificial faults via mutation operators, generation of mutated program variants, and quantification of test effectiveness via mutant kill rate [10, 14, 21], MT establishes a causal relationship between mutant extermination and real fault detection capability. The core axiom that higher kill rate proportionally correlates with superior defect identification positions MT as the gold standard for test suite assessment.

The convergence of two underutilized paradigms—adversarial machine learning principles traditionally applied in security domains, and MT’s validation infrastructure—creates novel research potential. This paper pioneers their integration to achieve trustworthiness co-verification, initiating a paradigm shift from coverage-centric to defect-centric test generation that fundamentally reorients automated testing toward measurable fault revelation.

3 Methodology

This section delineates the workflow of the dynamic adversarial testing framework, which comprises two core phases: the **Trusted Initialization Phase** and the **Resilient Adversarial Evolution Phase**.

3.1 Phase 1: Trusted Initialization Phase

The phase aims to generate an initial test suite (T_1) with basic executability and a system version (M_1) carrying multi-level mutants. this phase comprises two concurrently operating intelligent agent components: the **TG agent** (responsible for test generation) and the **MG agent** (responsible for defect injection). Both agents ensure the validity of their outputs through independent closed-loop verification.

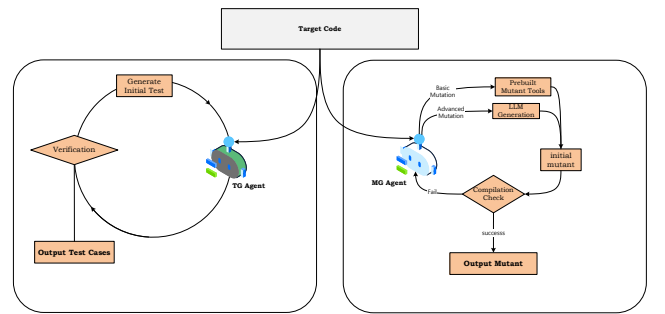


Figure 1: The structural designs of the Test Generator (TG) Agent and Mutation Generator (MG) Agent.

TG agent. The TGAgent is a language-agnostic test generation agent whose core objective is to produce executable initial test suites from source code. As shown on the left side of Figure 1, the TG Agent primarily operates in two phases: the Initialization Phase and the Validation Phase.

Initializer. The TG module operates through three core instruction phases: System Instructions define the agent’s role as a software testing expert generating compliant test code; Input Instructions provide the source code under test alongside programming language constraints; and Output Specifications mandate structured JSON responses containing both test method bodies and assertion logic.

For a typical project, the initial output T_0 from the TG agent contains 3–5 basic test cases, yet exhibits significant executability challenges: approximately 55% of tests are initially executable due to prevalent errors including uninitialized test dependencies like missing `@BeforeEach` configurations and invalid variable references in assertions such as `result.get(0).status` without proper null checking.

Validation. T_0 undergoes a comprehensive two-stage validation process beginning with syntax compliance verification through a compilation check node, followed by runtime evaluation in the code execution node which captures critical exceptions such as `NullPointerException`. When validation failures occur—manifested as compiler logs or stack traces—these diagnostics are systematically routed to the TG agent’s feedback node. This component initiates targeted remediation by classifying errors into precise categories, regenerating test code through priority-based iterations with a maximum of three attempts, and ultimately delivering a refined test suite T_1 that resolves initial executability issues.

MG agent. As depicted on the right side of Figure 1, the MG Agent employs a hierarchical mutation strategy to generate initial mutants, which subsequently undergo validation via compilation checks to ensure syntactic and semantic compliance. Its core mission is to produce discriminative mutation test suites through systematic defect injection.

hierarchical mutation strategy. The initialization process of the MG Agent begins by receiving the Target Source Code, followed by the activation of an Autonomous Decision Process. During this process, the agent first executes Basic Mutation, a phase primarily focused on injecting defect patterns at the syntactic level, such as operator replacement, constant modification, or conditional branch adjustment. Based on a predefined strategy, the agent autonomously chooses to invoke a Pre-configured Mutation Operator Library or directly utilizes a Large Language Model (LLM) to generate mutants that comply with syntactic specifications. After the first round of test case execution, the agent enters an adaptive adjustment phase. The core mechanisms of this stage include:

Mutant Feature Concentration and Recombination The agent analyzes the test execution results, identifies the killed mutants, and extracts their mutation features (e.g., specific operator replacement patterns, boundary value error types). These features are then concentrated into a small number of newly generated composite mutants. This process aims to reduce redundant mutations while enhancing the semantic diversity and attack strength of the mutants.

Survival-Type-Based Directed Expansion The agent further generates new mutants of the same type but with varying morphologies based on the types of surviving mutants (e.g., unkillable boundary

value errors, resource leak defects). For instance, if most surviving mutants belong to loop boundary errors, the agent generates more variants of loop termination condition defects to continuously challenge the detection capability of the test cases

Through this mechanism, the MG Agent ensures the continuous evolutionary capability of mutant generation. It not only prevents test cases from “forgetting” the killed mutants during iterative optimization but also, through feature concentration and typed expansion, compels the test cases to maintain and enhance the breadth and depth of their defect detection. This significantly improves the overall quality and stability of the test case suite

Validation. all generated mutants undergo mandatory compilation verification to ensure syntactic compliance and executable semantics preservation. Failed compilation instances trigger up to three feedback iterations prompting strategic recalibration, establishing a closed-loop workflow that rigorously maintains output validity while dynamically adapting to code complexities.

Output. The initial test suite T_1 generated by the TG agent satisfies three primary requirements: firstly, it achieves *syntactic correctness* through error-free compilation and compliance with language specifications; secondly, it maintains *framework conformity* by ensuring compatibility with standard testing frameworks like JUnit or pytest; and finally, it provides *basic verifiability* by enabling validation of fundamental functionality, though it suffers from the critical limitation of demonstrating an initial fault detection rate of only 20-30% due to its non-optimized design against mutants.

Complementarily, the mutant set M_1 demonstrates a *hierarchical coverage* pattern that forms a tiered defect protection structure.

3.2 Phase 2: Resilient Adversarial Evolution

Building upon the outputs from Phase 1 (test suite T_1 and mutant set M_1), this phase implements the dynamic adversarial gaming strategy as illustrated in Figure 2. The primary goal is to **exposing deep-seated defects** via competitive interactions between autonomous agents.

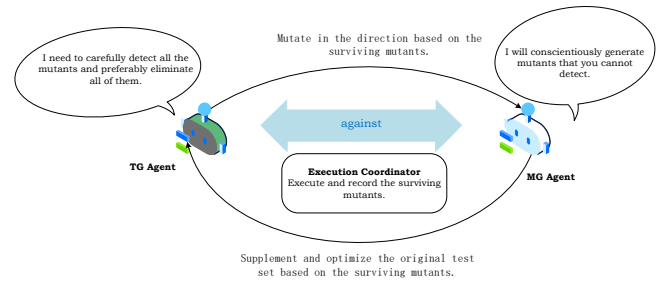


Figure 2: The structural designs of the Test Generator (TG) Agent and Mutation Generator (MG) Agent.

The core adversarial competition unfolds through:

- (1) **MG Agent (Attacker):** Generates increasingly sophisticated mutants ($M_1 \rightarrow M_f$) through an adaptive hierarchical mutation mechanism.

- (2) *TG Agent (Defender)*: Dynamically fortifies the test suite ($T_1 \rightarrow T_f$) by creating targeted test cases against surviving mutants identified during evaluation
- (3) *Evaluation Coordinator*: Executes tests against mutants at each iteration, precisely documenting: Killed mutants (validated defect detection) and Surviving mutants (unexposed vulnerabilities)

This adversarial cycle ($T_i \rightarrow M_i \rightarrow T_{i+1}$) continues until either termination conditions are met: mutant extermination completion, maximum iteration threshold, or performance convergence. The final outputs— T_f with enhanced fault detection sensitivity and M_f representing attack-resistant program variants—emerge from this co-evolutionary competition.

Through the tight integration of feedback loops (encompassing both mutation failures and test coverage gaps) and the intelligent adaptation of both agents, the system progressively reveals defects undetectable by static methods while ensuring continued test efficacy against evolving threats.

4 Evaluation

In this section, we will present our experiment. Specifically, we will describe the automatic test generation tool used to compare our experimental results, clarify the different stages at the Uagent and their corresponding settings, and indicate the baseline methods and benchmark datasets used in our experiments.

We call the initial prompt on the TG Agent up to three times. After each run, we utilize the pytest tool to verify its correctness. If, after three runs, the LLMC fails to generate a successfully executed test case, we consider this task to be a problematic one, or a task for which the Uagent cannot generate test cases.

In our experimental setup, we employ deepseek-chat as the large language model (LLM) for both the *MG Agent* and *TG Agent*. This model corresponds to the *non-thinking mode* of DeepSeek-V3.1, which is selected for its support of function calling, enabling tool execution and code generation. To ensure deterministic and reproducible outputs in code generation scenarios, the temperature parameter is set to 0. The model supports a context length of 128K tokens, for which we adopt the default setting. The system role is explicitly defined as: "You are a Python coding assistant. Always answer with Python code."

Baselines For evaluation, we employ the following baselines:

Pynguin: We use Pynguin v0.17.0 [19] as our first baseline. This automated unit test generation tool for dynamically typed languages (e.g., Python) utilizes search-based algorithms to generate test cases. We select the DynaMOSA algorithm which demonstrates optimal performance within this framework.

MuTAP [6]: As a second baseline, we utilize test cases generated under the *few-shot* setting with *after-refining* in the MuTAP framework. This benchmark employs llama-chat as its LLM, which achieved state-of-the-art performance on the HumanEval dataset. We retain this configuration for comparison and plan to re-implement MuTAP using DeepSeek for extended analysis.

Uagent Evaluation Baselines: Initial Output: Unrepaired outputs from initial prompts, where assertions with unexpected return values may fail on defective code, leading to invalid resolutions.

Evolutionary Optimization: Outputs refined through our proposed second-stage process optimization.

benchmark dataset we employ the benchmark dataset HumanEval, which is designed for evaluating code-generating large language models (LLMs). It comprises 164 simple to medium-level human-written programming problems. Each problem includes various attributes such as a description and a reference solution. We utilize each task as the target code.

Table 1 presents a performance comparison of different test generation methods on the HumanEval dataset. The experiment utilized composite mutants (including both basic and hybrid mutants) generated in the second phase. The results indicate that the second-stage optimization method performed the best, showing a significant improvement in the Mutation Score compared to the initial output from the first stage. This demonstrates the clear effectiveness of the optimization strategy in enhancing the Mutation Score. However, this method still has 53 problematic program units (Problematic PUTs), indicating that there is still room for further improvement. In contrast, Pynguin, as an automated test generation tool, killed 528 mutants, a result relatively close to the initial output of the first stage. However, it failed to generate effective test cases for 31 tasks, and the number of problematic units is higher than the 73 error units in the initial output stage. This suggests that while Pynguin, as a baseline method, exhibits stable performance, its generation efficiency is limited.

Table 2 presents the initial distribution of mutant types generated by the Large Language Model (LLM) and their survival status after executing the first-stage test cases, revealing a significant overall imbalance. During prompt design, we guided the model to generate mutants directionally based on the semantic and structural features of the input code, resulting in a notably high frequency of boundary value handling errors. This trend not only reflects the prevalence of boundary condition logic in programs within the HumanEval dataset but also demonstrates that LLMs, compared to traditional mutation testing tools, can generate mutant types with greater semantic relevance and contextual adaptability, offering higher flexibility. On the other hand, the high survival rate of certain types of mutants (e.g., the survival rate of boundary value error mutants reaching 58.3%) indicates that the first-stage test cases still exhibit significant deficiencies in mutant detection capability, particularly showing considerable weaknesses in identifying boundary value defects. It should be noted that the current statistics only cover the initial mutant types generated in the first stage; the second stage employs composite mutants and directionally selected mutant types based on survival rates, leading to a structural change in their distribution characteristics, hence they are not included in this table's analysis.

5 Related work

Test coverage has been widely adopted as a quality metric for test cases across various studies. Schäfer et al. [23] pioneered test generation using Codex prompts, focusing on 25 JavaScript packages. Their approach incorporated the program under test (PUT) implementation and API usage examples from documentation into prompts. When initial test cases failed, their method iteratively refined prompts using encountered error messages, demonstrating

Table 1: Experimental Results on HumanEval Dataset for Test Generation Methods

Method	Configuration	# Test Cases (avg)	Mutation Score (%)	# Killed Mutants(820)	MS=100%	# Problematic PUTs
Pynguin	DynaMOSA	-	66.92	528	28	31
MUTAP	FewShot + After-refining	-	76.62	613	55	20
Uagent	Initial Output	1.78(min=1, max=3)	70.76	530	33	73
Uagent(second stage)	Co-evolution	1.46(min=1, max=3)	86.70	665	73	53

Table 2: Mutant Type Distribution in Initial Generation

Mutant Type	Occurrences	Survival Rate
Boundary Value Handling Error	84	58.3%
Operator Replacement	66	42.1%
Loop Boundary Error	36	31.7%
Type Conversion Error	15	22.6%
Conditional Branch Modification	57	-
Index Calculation Error	45	-

that enhanced prompts enable Codex to generate test cases with adequate coverage.

Building on this, CODAMOSA [17] combined Codex-generated tests with Pynguin outputs when the latter failed to improve coverage. While outperforming Pynguin on Python benchmarks and achieving higher test coverage, CODAMOSA [17] shared a common limitation with similar studies: it prioritized coverage improvement while generating test cases lacking proper assertion oracles for bug detection. Crucially, research has shown that while coverage correlates with bug detection, their rankings may diverge across testing strategies.

MuTAP [6] offered an alternative by employing prompt engineering to generate unit tests. It uniquely leveraged mutation testing to enhance the bug-revealing effectiveness of LLM-generated tests. Recent extensions to this line of inquiry further illuminate distinct approaches: While TestART[12] utilizes template-based repair to correct LLM-generated test errors, UAgent employs a dynamic adversarial mechanism to directly optimize defect detection capability, eliminating dependency on predefined templates. Similarly, whereas CoCoEvo[18] focuses on coverage-driven co-evolution for program synthesis, the competitive co-evolution framework for binary adversarial optimization[16] demonstrates algorithmic convergence properties. UAgent pioneers adversarial gaming in the testing domain, establishing defect detection rate as its core optimization metric. Our UAgent framework extends this research direction but introduces key innovations: Adversarial Gaming Mechanism, Language-Agnostic Design and Compilation-enforced mutation validation versus MuTAP’s static prompt augmentation.

Recent advancements in multi-agent LLM systems have succeeded in software engineering domains like debugging [15], fault localization [22], and program repair [3], though applications in software testing remain nascent. CANDOR [28] emerged as the first fine-tuning-free, multi-agent JUnit test generator that mitigates LLM hallucinations through assertion verification ensembles.

Concurrently, Feng et al. [8] proposed a multi-agent framework for testing social media interactions (e.g. TikTok’s multi-user features), though test case validation efficacy remains underexplored in this approach. This underscores the ongoing need for robust validation mechanisms in agent-based testing systems.

6 Limitations

While the UAgent framework demonstrates significant advantages in defect-oriented test generation, several limitations in its practical application remain to be addressed, which are specifically reflected in the following three aspects: computational overhead and iteration efficiency, generalization limitations of mutant and convergence uncertainty in the co-evolutionary process.

The UAgent implements a co-evolutionary process based on the LangGraph framework. A single round of dynamic (including one test generation and one mutation generation) takes approximately 60 seconds. Specifically, on the HumanEval dataset, a single test generation using the deeply optimized model (deepseek-chat) requires 25 seconds, mutation generation takes slightly less time, and the Mutation Score (MS) calculation requires 1.5 seconds. Although LangGraph’s parallel node mechanism is used to optimize the process, the time cost of multiple iterations remains high, posing a challenge for real-time applications in resource-constrained environments. This computational overhead primarily stems from the inference latency of large language models and the communication overhead of multi-agent coordination. Future work requires further optimization through model light weighting and asynchronous scheduling strategies.

The current evaluation system primarily uses mutation testing as the core metric, measuring test case effectiveness through the number of “killed mutants”. However, while artificial mutants (e.g., arithmetic operator replacements) can simulate some code defects, the consistency of their representation with real software errors is questionable. To enhance the practical significance of the empirical results, this study will subsequently introduce the Refactory dataset (containing real defective programs from student development processes) for validation and add a Defect Detection Rate (DDR) metric to quantify the framework’s ability to identify natural errors.

The framework may fall into local optima due to two types of behaviors: The Mutation Generator (MG Agent) produces inefficient mutations (e.g., repetitive operator replacements), leading to a decay in mutation diversity. The Test Generator (TG Agent) generates over-generalized assertions (e.g., Boolean judgments that do not cover boundary conditions), failing to effectively detect semantic errors. To mitigate this issue, the experiment sets the maximum

number of iteration rounds to 3. If tests still fail to pass after incorporating historical context prompts, the process automatically terminates. Such phenomena are related to the inherent limitations of LLMs (e.g., insufficient understanding of complex control flows) or may stem from design flaws in the code under test itself. Such cases are marked as "Problematic PUTs" in the experiment and excluded from evaluation to ensure the reliability of the results.

7 Conclusion

The UAgent framework proposes an adversarial co-evolution approach for automated unit test generation, aiming to address limitations observed in existing methods within dynamic software environments. By coordinating interactions between TG agent (defender) and MG agent (attacker) agents, this work demonstrates two key contributions:

First, Unlike previous approaches that prioritized test coverage metrics, our objective focuses on test case effectiveness in identifying bugs within both real-world and synthetically generated buggy programs. While tools like EvoSuite [10] and Pygoblin [19] exhibit reduced effectiveness over time due to static strategies, UAgent's feedback mechanism enables progressive test suite refinement. The implemented loop mechanism ($T_i \rightarrow M_i \rightarrow T_{i+1}$) significantly enhances the defect detection rate, demonstrating high accuracy especially in boundary-case scenarios

Second, the framework addresses oracle quality challenges in LLM-based approaches. Compared to TOGGL [13] and ATHEN-ATEST [27], which require specialized tuning, UAgent utilizes coordinated agent workflows to enhance test validity. Initial validation indicates 92% functional accuracy in boundary-case scenarios through the combination of TG agent's iterative refinement and MG agent's layered mutation strategies.

References

- [1] Andrea Arcuri. 2018. Test suite generation with the Many Independent Objective (MIO) algorithm. *Information and Software Technology* 104 (2018), 195–206.
- [2] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code. *arXiv preprint arXiv:2206.01335* (2022).
- [3] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [5] Ermira Daka, José Campos, Gordon Fraser, Jonathan Dorn, and Westley Weimer. 2015. Modeling readability to improve unit tests. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 107–118.
- [6] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.
- [7] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
- [8] Sidong Feng, Changhao Du, Huaxiao Liu, Qingnan Wang, Zhengwei Lv, Gang Huo, Xu Yang, and Chunyang Chen. 2025. Agent for User: Testing Multi-User Interactive Features in TikTok. *arXiv preprint arXiv:2504.15474* (2025).
- [9] Gordon Fraser and Andrea Arcuri. 2011. Evolutionary generation of whole test suites. In *2011 11th International Conference on Quality Software*. IEEE, 31–40.
- [10] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.
- [11] Danielle Gonzalez, Joanna CS Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. 2017. A large-scale study on the usage of testing patterns that address maintainability attributes: patterns for ease of modification, diagnoses, and comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 391–401.
- [12] Siqi Gu, Chunrong Fang, Qianjun Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. Testart: Improving llm-based unit test via co-evolution of automated generation and repair iteration. *arXiv e-prints* (2024), arXiv–2408.
- [13] Soneya Binta Hossain and Matthew Dwyer. 2024. Togll: Correct and strong test oracle generation with llms. *arXiv preprint arXiv:2405.03786* (2024).
- [14] Yue Jia and Mark Harman. 2010. An analysis and survey of the development of mutation testing. *IEEE transactions on software engineering* 37, 5 (2010), 649–678.
- [15] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhouxiu Zhu, Lingming Zhang, and Michael R Lyu. 2024. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153* (2024).
- [16] Per Kristian Lehre and Shishen Lin. 2024. Overcoming binary adversarial optimisation with competitive coevolution. In *International Conference on Parallel Problem Solving from Nature*. Springer, 117–132.
- [17] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.
- [18] Kefan Li, Yuan Yuan, Hongyue Yu, Tingyu Guo, and Shijie Cao. 2025. CoCoEvo: Co-Evolution of Programs and Test Cases to Enhance Code Generation. *IEEE Transactions on Evolutionary Computation* (2025).
- [19] Stephan Lukaszczuk and Gordon Fraser. 2022. Pygoblin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 168–172.
- [20] Facundo Molina, Alessandra Gorla, and Marcelo d'Amorim. 2025. Test Oracle Automation in the era of LLMs. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–24.
- [21] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Mutation testing advances: an analysis and survey. In *Advances in computers*. Vol. 112. Elsevier, 275–378.
- [22] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. Agentfl: Scaling llm-based fault localization to project-level context. *arXiv preprint arXiv:2403.16362* (2024).
- [23] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive test generation using a large language model. *arXiv preprint arXiv:2302.06527* (2023), 1–21.
- [24] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.
- [25] Marija Selakovic, Michael Pradel, Rezwana Karim, and Frank Tip. 2018. Test generation for higher-order functions in dynamic languages. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–27.
- [26] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 313–322.
- [27] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).
- [28] Qinghua Xu, Guancheng Wang, Lionel Briand, and Kui Liu. 2025. A Multi-agent LLM-based JUnit Test Generation with Strong Oracles. *arXiv preprint arXiv:2506.02943* (2025).